

Bloom Filters & Probabilistic Data Structures

Phase 3 · Advanced · Estimated time: 1.5 hours

1. Concept From First Principles

Chapter 5 mentioned, in passing, that Cassandra uses bloom filters to skip SSTables that definitely don't contain a key. This chapter explains exactly what a bloom filter is and why trading a small, controlled chance of being wrong for a dramatic reduction in memory usage is often the correct engineering choice at scale — the first of several **probabilistic data structures** this chapter covers, all built on the same underlying idea.

Bloom filters: fast, memory-efficient 'have I seen this before?'

A **Bloom filter** answers one specific question extremely efficiently: "is this item possibly in the set, or definitely not in the set?" It's a fixed-size bit array (all bits start at 0) paired with several independent hash functions. To **add** an item, you run it through each hash function, and set the bit at each resulting position to 1. To **check** if an item might be present, you run it through the same hash functions and check whether *all* the resulting bit positions are set to 1 — if even one is 0, the item is **definitely not** in the set (guaranteed, no ambiguity). If all are 1, the item is **probably** in the set — but this could be a **false positive**, where those specific bits happened to be set by the combination of some other, different items entirely. Critically, a Bloom filter never produces a false negative: if it says "not present," that's always correct.

Backend engineer analogy

Imagine checking whether a username is already taken by looking it up in a giant, precise database index — accurate, but the index itself might be gigabytes for a billion usernames. A Bloom filter is like a tiny, in-memory 'maybe taken, maybe not' cache in front of that index: if it says 'definitely not taken,' you skip the expensive database lookup entirely and let the signup proceed instantly; if it says 'possibly taken,' you fall back to the real, authoritative check. Most lookups (the vast majority of usernames someone tries ARE actually available) get the fast path.

The false-positive rate is tunable: using more bits per item and more hash functions reduces false positives, at the cost of more memory — but even at a very low false-positive rate (under 1%), a Bloom filter typically needs less than 10 bits per item, regardless of how large or complex the actual items being tracked are, which is the entire appeal: you get a massive memory reduction compared to storing the actual items, in exchange for accepting a small, controllable, one-directional error rate.

HyperLogLog: counting distinct items without storing them

HyperLogLog solves a different problem: estimating the number of *distinct* items in a huge stream ("how many unique visitors hit our site today?") without storing every item seen. A naive exact approach would need a set containing every unique visitor ID, potentially gigabytes of memory for a high-traffic day. HyperLogLog uses clever probabilistic bit-pattern observations (roughly: tracking the longest run of leading zeros seen in hashed values, which statistically correlates with the number of distinct items) to estimate cardinality using a tiny, fixed amount of memory (often just a few kilobytes), typically accurate

to within about 2% error — an extraordinary trade of a small, known error margin for a massive reduction in memory that scales to essentially any input size.

Count-Min Sketch: approximate frequency counting

Count-Min Sketch solves yet another related problem: estimating how *many times* each item has appeared in a stream ("how many times has this product page been viewed today?") without keeping an exact counter per item, which would require memory proportional to the number of distinct items. It uses a small 2D array of counters and multiple hash functions (conceptually similar in spirit to a Bloom filter's structure), and always *overestimates* counts (never underestimates), with the amount of overestimation controllable by the structure's size — useful for finding "heavy hitters" (the most frequently occurring items) in a massive stream without per-item exact bookkeeping.

The unifying theme across all three structures, and the reason they belong in one chapter together: each one deliberately accepts a small, bounded, well-understood error rate in exchange for a dramatic and predictable reduction in memory usage, which is often exactly the right trade at genuine scale. The interview-relevant skill isn't memorizing the internals of every probabilistic data structure that exists; it's recognizing the *shape* of a problem — "do I need an exact answer, or would a fast, small, slightly-approximate answer genuinely be good enough here?" — and knowing that a purpose-built structure exists for the common versions of this question (membership, distinct count, frequency).

2. Real-World Example / Case Study

Flipkart's or Noon's username/email availability check during signup is a classic Bloom filter use case: a small, in-memory Bloom filter can instantly say "definitely available" for the vast majority of attempted usernames, falling back to a real database check only for the rare "possibly taken" cases, meaningfully reducing database load on a high-traffic signup flow.

A Swiggy or Talabat analytics dashboard showing "X unique customers ordered from restaurants in Dubai today" is a natural HyperLogLog application — computing this exactly across millions of daily orders would require holding every unique customer ID in memory, while HyperLogLog gives a close-enough-for-a-dashboard estimate using a tiny, fixed memory footprint that doesn't grow with traffic.

Razorpay's or PhonePe's fraud-detection pipeline (Chapter 7's stream-processing context) might use a Count-Min Sketch to identify unusually high-frequency transaction patterns from a specific device/IP/account combination in near-real-time, flagging "heavy hitters" for deeper investigation without needing exact per-key counters across an enormous, continuously flowing transaction stream.

3. Diagram

Bloom filter -- bit array of size 16, 3 hash functions:

```
ADD "razorpay": hash1->2, hash2->7, hash3->11
               [0,0,1,0,0,0,0,1,0,0,0,1,0,0,0,0]  <- bits 2, 7, 11 set to 1
```

```
CHECK "phonepe": hash1->2, hash2->9, hash3->11
                  bit 2 = 1, bit 9 = 0, bit 11 = 1
                  -> bit 9 is 0 -> DEFINITELY NOT in the set (guaranteed correct)
```

```
CHECK "swiggy": hash1->2, hash2->7, hash3->11
                  all three bits are 1 -> POSSIBLY in the set
                  (could be a TRUE positive, or a FALSE positive from other items
                   that happened to set the exact same 3 bits)
```

4. Trade-offs Table

Structure	Answers	Error type	Typical use
Bloom filter	Is this item in the set?	False positives possible; never false negatives	Skip expensive lookups, cache/DB pre-checks (Cassandra SSTables)
HyperLogLog	How many distinct items?	~2% estimation error	Unique visitor counts, distinct-value dashboards
Count-Min Sketch	How many times has this item appeared?	Overestimates, never underestimates	Heavy-hitter detection, approximate frequency counts

5. Common Interview Questions

Q1: "Explain how a Bloom filter works, and why it can have false positives but never false negatives."

Model approach: Walk through the add/check process with hash functions and bit positions, and explain that a false positive happens when other items collectively set the same bits, while a false negative is impossible because any bit an item set during ADD stays set forever — if a bit needed for a real member were 0, that member could never have been added.

Q2: "Design a system to check if a URL has already been crawled, given billions of URLs."

Model approach: Propose a Bloom filter as a fast, memory-efficient first check before hitting a real crawled-URL store, explaining the memory savings versus storing every URL in an exact set, and acknowledging the small false-positive risk (occasionally skipping a URL that wasn't actually crawled) as an acceptable trade-off for this use case.

Q3: "How would you estimate the number of unique users in a day without storing every user ID?"

Model approach: Recommend HyperLogLog, explain its fixed, tiny memory footprint regardless of input volume, and state the typical ~2% error margin as an acceptable trade for a dashboard/analytics use case that doesn't need exact precision.

Q4: "When would you NOT use a probabilistic data structure, even at large scale?"

Model approach: Name a case requiring exact correctness — a financial balance, an exact inventory count, anything where an error has real business or legal consequences — and explain that probabilistic structures are a deliberate trade of a small error for efficiency, never appropriate where that error isn't tolerable.

6. Practice Exercises

- 1 Design a Bloom-filter-backed username availability check for a signup flow, including what happens on a 'possibly taken' result.
- 2 Explain why a Bloom filter's false-positive rate decreases as you add more bits per item and more hash functions, and what the cost of that improvement is.
- 3 Propose where in a food delivery app's backend you'd use HyperLogLog and where you'd use Count-Min Sketch, with justification for each.

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"Bloom Filter System Design High Level Design"	Solid system-design-focused explanation of Bloom filters.
English	"Bloom Filter and Probabilistic Data Structure in detail Scalable Thinking"	Covers Bloom filters alongside other probabilistic structures.
Hindi	"What is a bloom filter ? How does it work ? Hindi"	Hindi explanation of Bloom filter mechanics with examples.
Hindi	"System Design Complete Course 2026" (Hindi + English, 10+ hrs)	Covers probabilistic data structures as part of a full bilingual course.

8. Key Takeaways

- Bloom filters answer 'definitely not present' or 'possibly present' using a tiny bit array and multiple hash functions.
- Bloom filters never produce false negatives, only (tunable) false positives.
- HyperLogLog estimates distinct-item counts using a fixed, tiny memory footprint, typically within ~2% error.
- Count-Min Sketch estimates item frequencies in a stream, always overestimating rather than underestimating.
- All three structures trade a small, bounded, well-understood error for dramatic memory savings at scale.
- The key interview skill is recognizing when an approximate answer is genuinely good enough for the use case.
- Never use these structures where exact correctness has real business, financial, or legal consequences.